

根号算法选讲（预习版）

Part I+II

wYYSZL

2026.8.10

Outline

主要讲分块思想，包括朴素的分块思想、块状数组、操作分块、莫队、根号分治等。
分块似乎也是技巧 > 算法，所以大多数内容都是题目。
假定大家对于入门组的内容已经比较熟练了，所以这部分内容会讲得快一点。
带星的题目是选做题目，但是多数也不会特别难。
欢迎大家挑战它们。

本节课内容

- ① 基础分块思想
- ② 块状数组
- ③ 根号分治
- ④ 莫队
- ⑤ 操作分块/根号重构
- ⑥ 杂题

目录

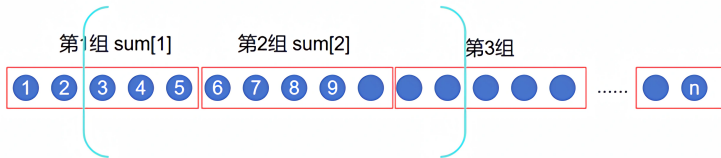
- ① 基础分块思想
- ② 块状数组
- ③ 根号分治
- ④ 莫队
- ⑤ 操作分块/根号重构
- ⑥ 杂题

基础分块思想

分块是暴力重构和懒标记的平衡。

分块的基本思想是，通过对原数据的适当划分，并在划分后的每一个块上预处理部分信息，从而较一般的暴力算法取得更优的时间复杂度。

当产生例如区间修改的操作时，遇到区间内的整块就打标记，遇到非整块就暴力修改。



基础分块思想

看这样一道例题：

例题

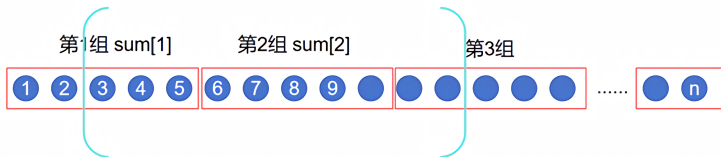
一个长度为 n 的序列 a 。 m 次操作，区间加或区间求和。

$$n, m \leq 10^5$$

当然我们可以用线段树，这里用来引入分块。

考虑我们将序列 a 按照每连续 $\sqrt{n}$ 个元素分一块，剩下的元素组成一块，并记录每个块的区间和 sum_x 。显然，总的块数为 $\frac{n}{\sqrt{n}} = \sqrt{n}$ 个（或者说 $\sqrt{n}$ 级别）。

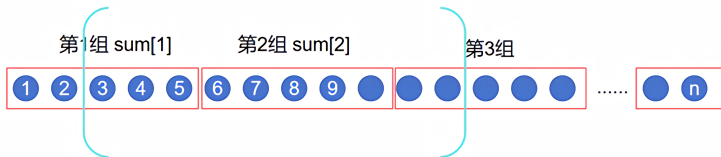
基础分块思想



首先看查询操作：

- 若 l 和 r 在同一个块内，直接暴力求和即可，复杂度为块长度 $O(\sqrt{n})$ 。
- 若 l 和 r 不在同一个块内，则答案由三部分组成： l 所在的不完整块，中间几个完整块，以 r 结尾的不完整块。对于不完整的块，仍然采用上面暴力计算的方法，对于完整块，则直接利用已经求出的 sum_i 求和即可。这种情况下，复杂度为两倍块长 + 总的块个数，于是 $O(\sqrt{n})$ 。

基础分块思想



接下来是修改操作，其实非常类似：

- 若 l 和 r 在同一个块内，直接暴力修改即可，块长为 $\sqrt{n}$ ，因此复杂度为 $O(\sqrt{n})$ 。注意，此时要修改对应块的 sum ，因为查询要用到。
- 若 l 和 r 不在同一个块内，则需要修改的也是三部分。对于不完整的块，仍然是暴力修改每个元素的值并改 sum ，对于完整块，则直接修改 sum_i 即可。这种情况下，复杂度仍然为 $O(\sqrt{n})$ 。

基础分块思想

这样，我们就在 $\mathcal{O}(m\sqrt{n})$ 的时间内解决了这个问题。

现在问题来了，为什么我们要用 $\sqrt{n}$ 这样的块长？

关键在于平衡复杂性以获得最优的总复杂度。我们设块长为 s ，则修改的复杂度为 $\mathcal{O}(2s + \frac{n}{s})$ （两个块长 + 块个数）。我们忽略常数，即 $\mathcal{O}(s + \frac{n}{s})$ 。利用基本不等式，有：

$$s + \frac{n}{s} \geq 2\sqrt{s \times \frac{n}{s}} = 2\sqrt{n}$$

当仅当 $s = \frac{n}{s}$ 时取最值，即 $s = \sqrt{n}$ 。

这种分析技巧应当掌握，有些题目中最佳块长不再是 $\sqrt{n}$ 而需要具体分析。

（如果没学过基本不等式，可以简单理解为积一定时，差越小和越小，或者直接记住两个相等最优也可以）

基础分块思想 - 分块的优势

这道题可以线段树，没有看出分块的优势。那我们为什么要用分块？

分块的主要作用有两个，一是**平衡复杂度**，二是维护一些 $\log$ 数据结构无法维护的信息。

- 对于区间加法，单点查询，一般的思路是使用树状数组维护。此时修改和查询的复杂度均为 $\log n$ 。分块的优势在于它可以让修改和查询当中的任意一个变为 $\mathcal{O}(1)$ ，另一个变为 $\sqrt{n}$ 。当询问或修改的次数非常多时，如莫队二次离线算法中的 $\mathcal{O}(n)$ 次区间修改， $\mathcal{O}(n\sqrt{n})$ 次单点查询，就可以使用分块平衡复杂度，做到非常优秀的 $n\sqrt{n}$ 。
- 当遇到无法快速合并和快速删除的信息时，传统的线段树就失效了。但分块仍然可以做到优秀的复杂度。

相当抽象，我们之后会有例题。

目录

- ① 基础分块思想
- ② 块状数组
- ③ 根号分治
- ④ 莫队
- ⑤ 操作分块/根号重构
- ⑥ 杂题

块状数组

很奇怪的名称。似乎没有一个严格定义。

块状数组的含义大概是，在一个块内维护这个块内的信息。通常这个信息量比较大而不是 $\mathcal{O}(1)$ 的。比如维护这个块内的全部某种特征的值。一般信息量只要控制在 $\sqrt{n}$ 之内即可。

我们具体看例题。

例题 A

A - 教主的魔法

给定长度为 n 的整数序列 $a_1, \dots, a_n$, 支持两种操作:

- 区间加: $[l, r]$ 内每个数增加 w ;
- 区间查询: 求 $[l, r]$ 中满足 $a_i \geq c$ 的元素个数。

输出所有查询答案。

$n \leq 10^6$, $q \leq 3000$ 。

线段树与分块

为什么不能直接用线段树？

原因在于， $\geq c$ 这样的信息是没有办法快速合并的。也就是说，如果知道左右儿子的信息，我们没法快速知道父亲的信息。同时，我们也无法快速地直接从修改中得到本节点的答案。

而分块不存在这样的问题。分块可以接受暴力的“合并”，对于这道题就是允许暴力地重新排序。这使得分块的适用范围远多于线段树等数据结构。

此外

分块可以看作只有两层的多叉线段树，因而线段树可以实现的操作都可以用分块以较劣的复杂度实现。有时由于分块的小常数，时间上分块甚至优于线段树。

例题 B

B - 作诗

给定 n 个不大于 c 的正整数 $a_1 \dots a_n$ 和 m 组询问, 每次问 $[l, r]$ 中有多少个数出现正偶数次。

强制在线, $1 \leq n, c, m \leq 10^5$

例题 C

C - 弹飞绵阳

给定长度为 n 的数组 $k_0, \dots, k_{n-1}$ ($k_i \geq 1$)，定义从位置 i 出发，每次跳到 $i + k_i$ ，若越界则停止。支持两种操作：

1. 查询：给定 j ，输出从 j 出发到停止所需的跳跃次数；
2. 修改：给定 j 和 r ，将 k_j 改为 r 。

数据规模： $n \leq 2 \times 10^5$ ， $m \leq 10^5$ 。

目录

- ① 基础分块思想
- ② 块状数组
- ③ 根号分治
- ④ 莫队
- ⑤ 操作分块/根号重构
- ⑥ 杂题

根号分治

根号分治本质上是一种 按规模大小分类讨论的思想而非分治算法。这里的“分治”表示分类而不是通常的“分而治之”的含义。对于规模为 x 的问题，如果我们能在 $\mathcal{O}(x)$ 和 $\mathcal{O}(\frac{n}{x})$ 的时间内解决，可以考虑根号分治：

- $x \leq \sqrt{n}$ 时使用 $\mathcal{O}(x)$ 的算法
- $x > \sqrt{n}$ 时使用 $\mathcal{O}(\frac{n}{x})$ 的算法

类似前面的，这相当于寻找 x 和 $\frac{n}{x}$ 的较小值的最大值：显然，当 $x = \sqrt{n}$ 时， $\min(x, \frac{n}{x})$ 取到最大值 $\sqrt{n}$ 。

说起来有点抽象，我们来看例题。

例题 D

D - Time to Raid Cowavans

给定长度为 n 的数组 $w_1, \dots, w_n$, 以及 p 个询问, 每个询问给出参数 (a, b) 。
定义一次询问的答案为:

$$\sum_{t \geq 0, a+tb \leq n} w_{a+tb}$$

依次输出每个询问的答案。

$n, p \leq 3 \times 10^5$ 。

例题 E

E - Till I Collapse

给定长度为 n 的数组 $a_1, \dots, a_n$ ($1 \leq a_i \leq n$)。对于每个 $k = 1, 2, \dots, n$, 定义 $f(k)$ 为将序列划分为若干个连续段, 使得每段内不同数值的个数不超过 k 所需的最少段数。输出 $f(1), f(2), \dots, f(n)$ 。

$n \leq 10^5$ 。

例题 F

F - Regions

给定一棵 n 个节点的有根树（根为 1），每个节点有一个颜色 $c_u \in [1, R]$ 。

对于每个询问 (r_1, r_2) ，求满足以下条件的节点对 (u, v) 的数量：

- u 是 v 的祖先；
- $c_u = r_1, c_v = r_2$ 。

$n, Q \leq 2 \times 10^5, R \leq 2.5 \times 10^4$ 。4s，强制在线。

小节

最后实际看来，根号分治通常是在 **预处理**和 **询问**的复杂度之间寻找平衡，和一开始的定义有一定出入。

根号分治还有另一类应用。利用根号分治思想，我们不难证明这一点：

数论分块

对于固定的 n ， $\lfloor \frac{n}{i} \rfloor$ 只有 $\mathcal{O}(\sqrt{n})$ 个取值。

由是，我们可以快速计算形如：

$$\sum_{i=1}^n f(i)g\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

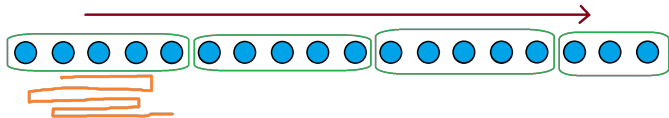
的式子。但这个东西超出了我们要讨论的内容，故掠过。

目录

- ① 基础分块思想
- ② 块状数组
- ③ 根号分治
- ④ 莫队
- ⑤ 操作分块/根号重构
- ⑥ 杂题

莫队

莫队是一个相当简单有效的离线做法，支持修改比较麻烦。



图：莫队

我们经常会遇到询问区间 $[l, r]$ 答案的问题。莫队的核心思想十分简单：我们现在已经知道 $[l, r]$ 的答案，我们希望每次通过暴力加入/删除点来扩展/缩小这个区间，得到下一个询问的答案。并按照一定顺序处理询问，使得时间复杂度最小。

下面是一些导出这种顺序的思路历程，初学可以跳过直接看后面的蓝框：

为了让指针移动距离尽可能少，我们可以将询问以某个端点为关键字排序。尽管该端点的移动距离均摊线性，但另一个端点的移动距离无法保证。

注意到两个端点的移动距离分别是 $\mathcal{O}(n)$ 和 $\mathcal{O}(nq)$ 。为了平衡复杂度，自然想到使用 **根号平衡** 的思想。如果将左右端点的移动距离都控制在 $n\sqrt{n}$ 以内，我们就得到了一个时间复杂度为 $\mathcal{O}(n\sqrt{n} \times k)$ 的优秀算法，其中 k 是指针移动的复杂度。

考虑如何分块。设块大小为 B 。我们将所有询问离线下来，并按照左端点 **块编号** 为第一关键字，右端点为第二关键字排序，按照该顺序处理所有询问的时间复杂度为 $\mathcal{O}\left(\frac{nq}{B} + qB\right)$ 。这是因为每个块内右端点的总移动距离不超过 n ，每次询问左端点的移动距离不超过 B 。假设 n, q 同级，取 $B = \sqrt{n}$ ，时间复杂度为 $\mathcal{O}(n\sqrt{n} \times k)$ 。

于是我们有了莫队算法：

莫队算法

将所有询问离线下来，并按照左端点所在块编号为第一关键字，右端点的编号为第二关键字排序。之后，暴力移动指针，从上一个询问到下一个询问。

我们分析复杂度。设块长为 $\sqrt{n}$ ，由于左端点不改变所在块时，右端点只会从左到右扫一次，所以右端点总的移动次数为 $\mathcal{O}(n\sqrt{n})$ 。左端点每次在块内移动或只移动一个块，不超过 $\mathcal{O}(\sqrt{n})$ ，总次数也是 $\mathcal{O}(n\sqrt{n})$ 。

设移动一次指针的代价为 k ，总的复杂度为 $\mathcal{O}(n\sqrt{n} \times k)$ 。

例题 G

G - 小 Z 的袜子

给定长度为 n 的整数序列 $a_1, \dots, a_n$, 以及 m 个询问, 每个询问给出区间 $[l, r]$ 。
从区间 $[l, r]$ 中等概率地随机选取两个不同位置 $i \neq j$, 求 $a_i = a_j$ 的概率。
 $n, m \leq 5 \times 10^4$, $a_i \leq n$ 。

莫队 - 实现

莫队在移动端点时的关键过程是加入/删除节点，且会复用。所以一个写法的建议是为加入/删除分别单独写函数，然后每次简洁调用即可。

```
1 void del(int pos);// 传入要删除的点编号
2 void ins(int pos);// 传入要加入的点编号
3 // l++ 和 ++l
4 while (l < query[i].l) del(l++);
5 while (l > query[i].l) ins(--l);
6 while (r < query[i].r) ins(++r);
7 while (r > query[i].r) del(r--);
```

图: 实现

不过也要注意有些题目是不能这样的。

例题 H

H - 异或序列

给定长度为 n 的整数序列 $a_1, \dots, a_n$, 常数 k 。 m 次询问, 每次给出 $[l, r]$, 求满足 $l \leq x \leq y \leq r$ 且

$$a_x \oplus a_{x+1} \oplus \dots \oplus a_y = k$$

的整数对 (x, y) 的数量。

数据规模: $n, m \leq 10^5$, $a_i, k \leq 10^5$ 。

类似上题。注意到 $a \oplus b = k$ 等价于 $a \oplus k = b$, 加入/删除一个点时, 统计窗口内与当前值异或和为 k 的值的出现次数。

例题 I

I - 作业

给定长度为 n 的序列， m 次询问给定 l, r, a, b 。求区间 $[l, r]$ 中介于 a, b 之间的数的个数，和介于 a, b 之间的，在区间内出现过的数值的个数。

说人话：求下面两个值：

- $ans_1 = |\{i \in [l, r] : a \leq a_i \leq b\}|$
- $ans_2 = |\{x \in [a, b] : \exists i \in [l, r], a_i = x\}|$

$$n, m \leq 10^5$$

* 例题 J

*J - 八云蓝自动机 I

给定长度为 n 的整数数组 A （初始值给定），以及长度为 m 的操作序列，每个操作是以下三种之一：

- 赋值： $A_x \leftarrow k$;
- 交换：交换 A_x 与 A_y ;
- 查询：输出当前 A_x 的值。

有 q 次询问，每次给出区间 $[l, r]$ 。对于每次询问，从初始数组 A 出发，依次执行操作 $l, l+1, \dots, r$ ，将所有查询操作输出的值累加，输出该累加和。

数据规模： $n, m, q \leq 2 \times 10^5$ 。

回滚莫队

我们说到，莫队的关键是如何快速加入/删除点来扩展/缩小区间，但并不是所有东西都能够快速删除（比如最值问题）。所以，我们希望能有一种方法解决这样的问题。这就是回滚莫队：

回滚莫队

将询问按照正常莫队的顺序排。在处理左端点在一个块内的所有询问时，先将左端点置于下一个块的开头，右端点为 $l-1$ 表示目前为空。

每次询问时，先将左端点退回到下一个块的开头，然后移动左右端点。这样没有删除操作而只有加入操作。

关键在于如何“退回到下一个块的开头”。

回滚莫队

有两种方法，一种方法是保存左端点在下一个块开头时的信息，稍微控制地暴力恢复。另一种方法是注意到虽然我们删除，但是可以撤销。

具体地，从第一个询问开始，先移动右端点，然后将左端点从“下一个块开头”移动。这样下一个询问时，直接反过来做所有你做过的操作（第二种方法），或者用记录下来的修改过的地方恢复（第一种方法），将左端点退回去，再移动右端点、左端点即可。关键是要先移动右端点。

关于撤销，猜你想搜：~~Command 设计模式。（完全没必要。）~~

注意到回滚莫队不能处理左右端点在一个块内的答案，需要暴力处理。这样复杂度不退化，依然是 $O(n\sqrt{n})$ 。

Hint

反过来，如果一个东西容易删除不容易加入，也可以反着使用回滚莫队。

例题 K

K - 【模板】回滚莫队

给定一个序列，多次询问一段区间 $[l, r]$ ，求区间中相同的数的最远间隔距离。

$$1 \leq n, m \leq 2 \cdot 10^5$$

模板题。

目录

- ① 基础分块思想
- ② 块状数组
- ③ 根号分治
- ④ 莫队
- ⑤ 操作分块/根号重构**
- ⑥ 杂题

操作分块/根号重构

操作分块是另一种技巧。一般用于各种数据结构题的非正解。

我们通常会遇到这样的题：给你一个东西，然后若干操作（修改/询问）。

操作分块的思想是，我们对这些操作进行分块，一个块结束时，将块内的修改操作一起执行，得到假设之后不再有修改时，下一个块内询问的答案，或者方便求出答案。然后对于下一个块内实际的询问，暴力看这个块内的每个操作对每个询问的影响，设块长 B 。复杂度 $\mathcal{O}(B^2)$ 。

听起来很抽象，我们来看例题。

例题 L

L - Xenia and Tree

给定一棵 n 个节点的树，初始时节点 1 为红色，其余节点为蓝色。

支持两种操作：

1. 将节点 v 染成红色；
2. 查询节点 v 到最近红色节点的距离。

数据规模： $n, m \leq 10^5$ 。

目录

- ① 基础分块思想
- ② 块状数组
- ③ 根号分治
- ④ 莫队
- ⑤ 操作分块/根号重构
- ⑥ 杂题

例题 M

M - [Violet] 蒲公英

给定长度为 n 的数组 a ，强制在线回答 m 个区间众数查询（若有多个，输出值最小的）。

$n \leq 4 \times 10^4$, $m \leq 5 \times 10^4$ 。

例题 N

N - 秃子酋长

给一个长为 n 的排列 $a_1, \dots, a_n$, 有 m 次询问, 每次询问区间 $[l, r]$ 内, 排序后相邻的数在原序列中的位置的差的绝对值之和。

$$1 \leq n, m \leq 5 \times 10^5$$

例题 O

O - 桥梁

给定一张 n 点 m 边的无向图，第 i 条边带权 w_i ，有 q 次如下两种操作：

- $w_x \leftarrow y$
- 查询从 x 开始，只通过 $\geq y$ 的边，可达点数量。

$1 \leq n \leq 5 \times 10^4$, $0 \leq m \leq 10^5$, $1 \leq q \leq 10^5$, $w_i, y \leq 10^9$

* 例题 P

P - 众数

给定长度为 n 的整数序列 a ，允许选择任意区间 $[l, r]$ 和任意整数 k ，将区间内所有数加上 k （操作恰好一次）。

求操作后序列众数的最大出现次数，并输出所有能达到该最大次数的众数取值。

$\sum n \leq 5 \times 10^5$ ，3s。可以扩到 6s。